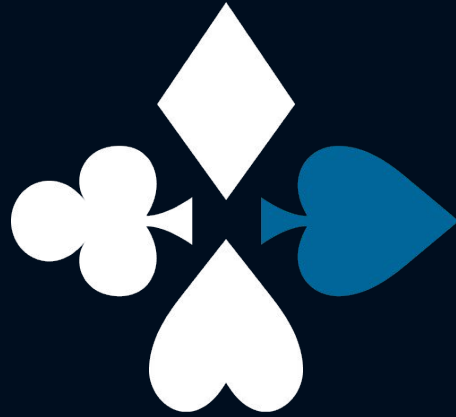




Pokerbots 2026

Lecture 6: Reinforcement Learning

Sponsors



CITADEL | CITADEL Securities





Announcements

GTO Wizard Challenge!

- Normal Heads-Up (Not Toss or Hold-em)
- Deadline: *Jan 30th*
- First Place: \$2,500
- Second Place: \$1,500
- Third Place: \$1,000
- pkr.bot/gto
 - Details on pkr.bot/piazza



Hackathon Tomorrow!

- Wednesday night (1/14)
- 32-124 or 34-501, 6pm -- LATE
- Show up and grind
- Dinner provided
- Snacks, fun, and games
- Prizes for challenges and those that stick around
- Details will be announced on Piazza

Snack/Drink Recs?

pkrfbot.com/snack

Make-up Progress Reports due Tuesday

- As per syllabus, if your team missed the Week 1 bot submission deadline, a progress report is required to earn credit
- Reports are half a page and must list team name and members
- Submit via email to pokerbots@mit.edu by 11:59pm Tuesday (Tomorrow)

Week 2 Bot Deadline

- Second pokerbot submission due Friday 1/16, 11:59PM EST on scrimmage server
- Mini-tournament 2 will occur shortly after

No Class Thursday

Cancelled in light of Hackathon the night before

No Class Monday

Happy MLK Day!



Giveaway Game!

Sum Game: pkr.bot/sum

- Submit one integer 0-9 (inclusive)
- Estimate the sum of all submitted integers (from everyone who fills out form)!
 - Feel free to guess a decimal value
- Winner decided by absolute distance from the true answer
 - Catch: your guess must be $\geq$ the true sum to be considered 😲
- Ties broken by earliest submission
- Prize: RedThunder K95 Mechanical Keyboard



Reinforcement Learning

From Thursday

- Supervised Learning

- Dataset has examples of correct behavior
- “Questions with answers”
- Prediction tasks

- Unsupervised Learning

- Dataset has no notion of good behavior
- Task is to find patterns to represent dataset
- Clustering, compression, generative modeling

- Reinforcement Learning (RL)

- Environment with feedback (rewards), but no correct target
- Sequential decision making, control, pokerbots!



Overview and Landscape

Reinforcement Learning Overview

Definition: Training an agent to make decisions by interacting with an environment to maximize some reward signal aggregated over many steps.

The task usually involves sequentially taking actions and observing their effects.

Examples:

- Most board/card/video games
- Control: robotics, self-driving cars
- Recommendation: advertisements
- Agent settings: interacting with software

Contrast with other types of Learning

Supervised and unsupervised learning analyze a fixed dataset and seek a model that closely aligns with that dataset.

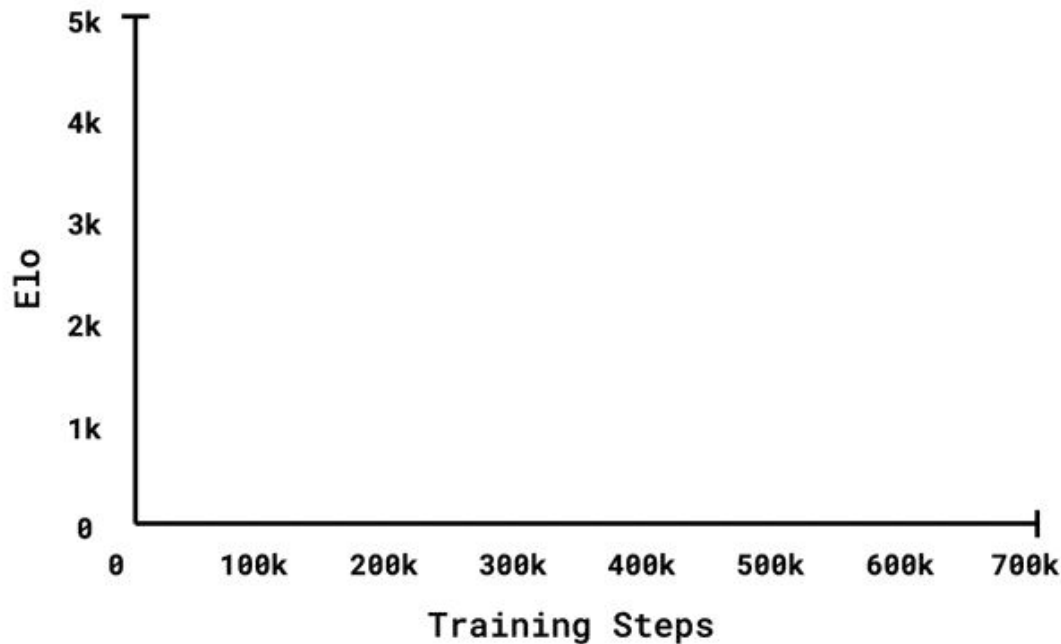
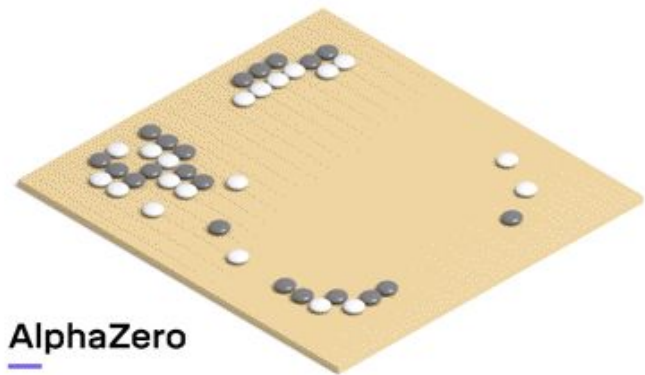
Reinforcement learning works *within* an environment and seek a behavior that performs well

- The data is the history of interactions with the environment
- Not necessarily fixed if more interactions can be made - sometimes deciding how to interact in order to collect more data is the hard part
- Sometimes the environment itself still needs to be studied, as this can help with navigating to get rewards

Successes

- AlphaZero: trained entirely from self-play
 - Beat best-in-world chess engine (stockfish) starting from only the rules of the game
- DeepMind “parkour” paper:
 - Inputs: terrain map, joint angles, angular velocities
 - Reward: forward progress

AlphaZero: from zero to mastery in four hours



Parkour



Why don't we all use reinforcement learning?

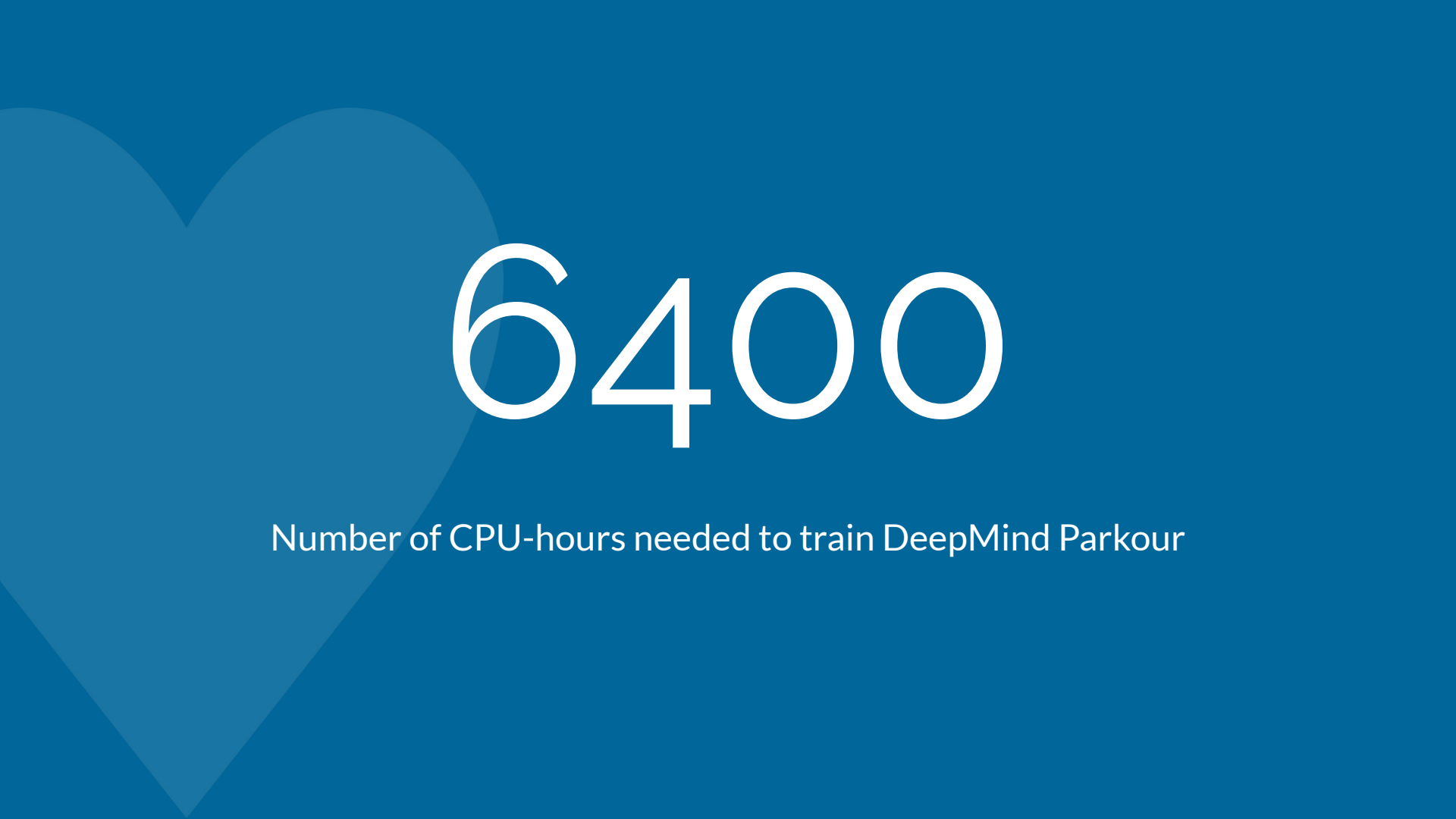
- Hard to train
 - Sensitive to parameters
 - Escaping local optima
- Sample inefficient
 - Complex environments require lots of exploration





5000

Number of processors needed to generate self-play games for AlphaZero's training



6400

Number of CPU-hours needed to train DeepMind Parkour

Struggles with multi-agent scenarios

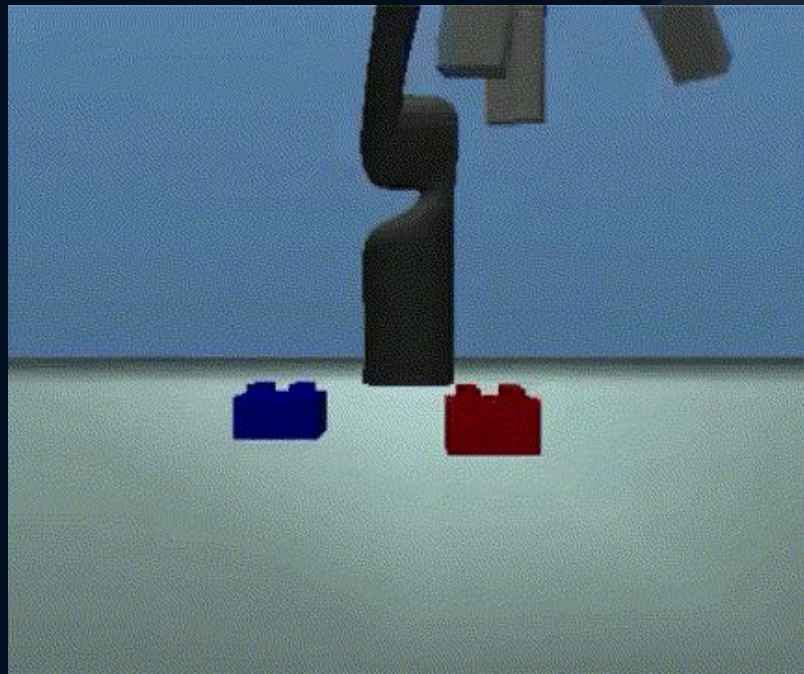
- In chess, if bot1 loses to bot2 and bot2 loses to bot3, then there is a good chance that bot3 is the strongest chess player
- Gives a clear route to *iterated improvement* with techniques such as *self-play*
- This is far from guaranteed in poker
- Game theoretic settings don't have the luxuries of supervised learning

Return to rock-paper-scissors

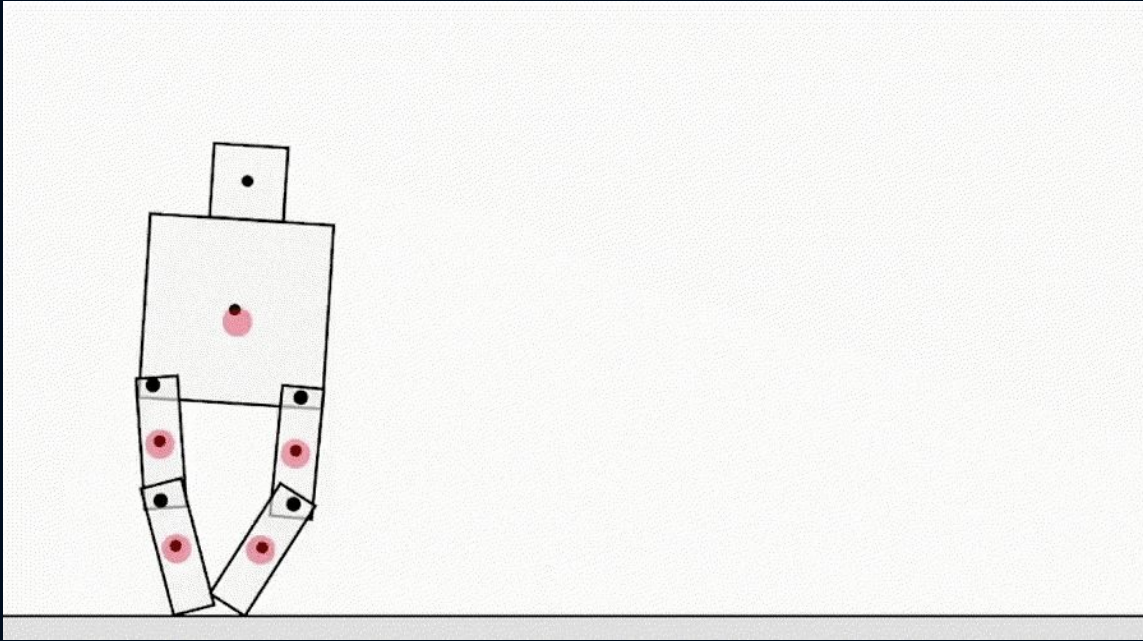
- When two reinforcement learning agents are trained against each other, they get very good at beating *each other*
- Risk of getting caught in a cycle without making meaningful improvements to performance
- A: rock $\rightarrow$ B: paper $\rightarrow$ A: scissors $\rightarrow$ B: rock $\rightarrow$
A: paper $\rightarrow$ B: scissors $\rightarrow$ A: rock...
- Cycles are predictable, which is undesirable

What to reward?

- Designing a good reward function is hard
- Luckily in poker, the job is done for us
- Specification gaming

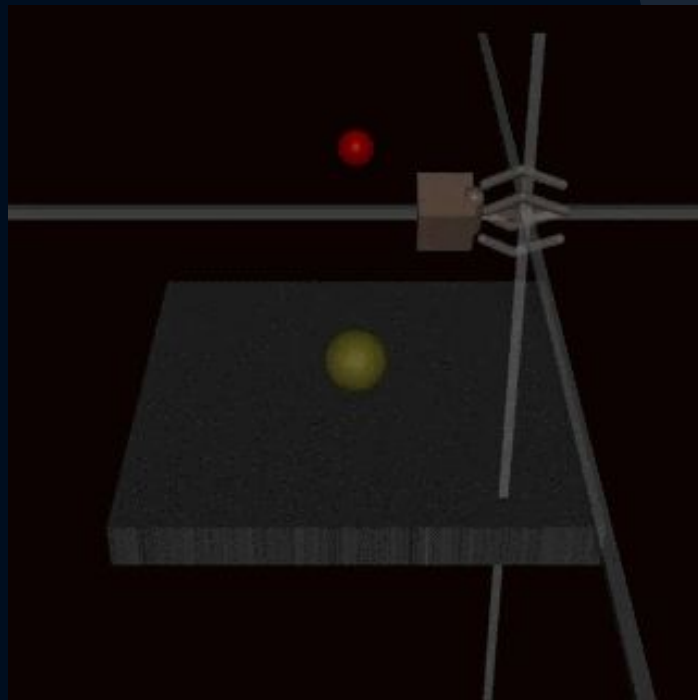


Common Code Bullet Win

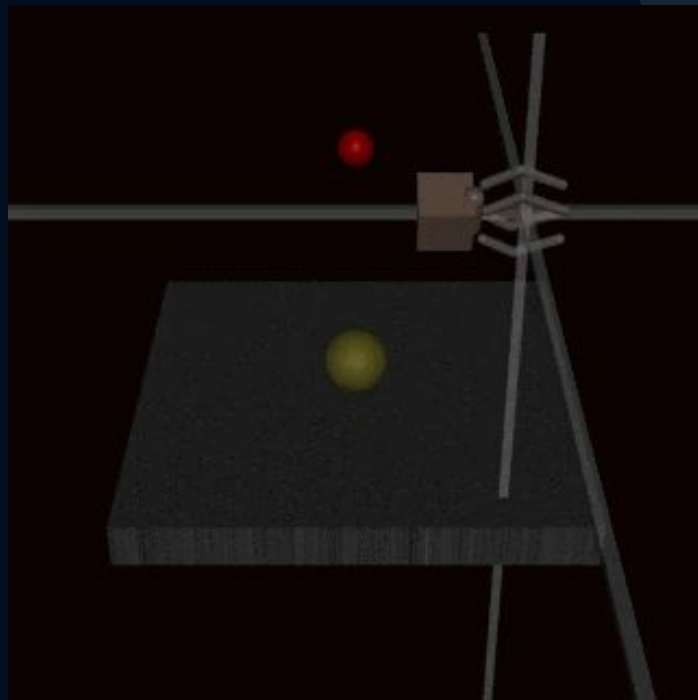
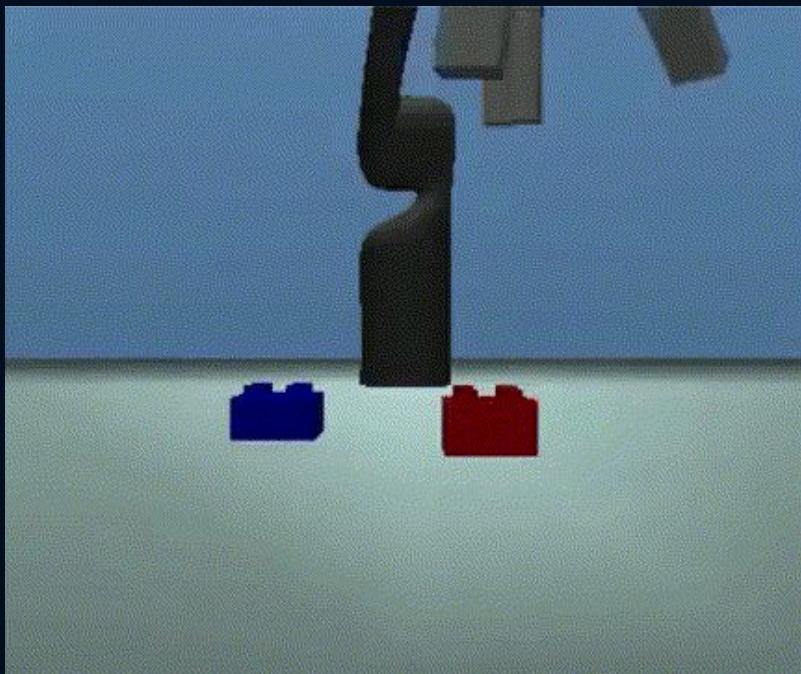


Specification Gaming

- Example: ChatGPT
- Reinforcement Learning from Human Feedback (RLHF)



Specification Gaming



Suppose we've considered all the warnings...

- Well-crafted reward function
 - Clear goal in mind for what we want
- Sufficient compute resources
- Handling multi-agent scenarios
- How do we train our policy?



Formalization

Problem Setting

- Agent: Learner/decision-maker.
- Environment: World the agent interacts with.
- State: Current situation.
- Action: Decision made by the agent.
- Policy: How our agent decides which action to take.
- Reward: Feedback for the action.

Trajectory:

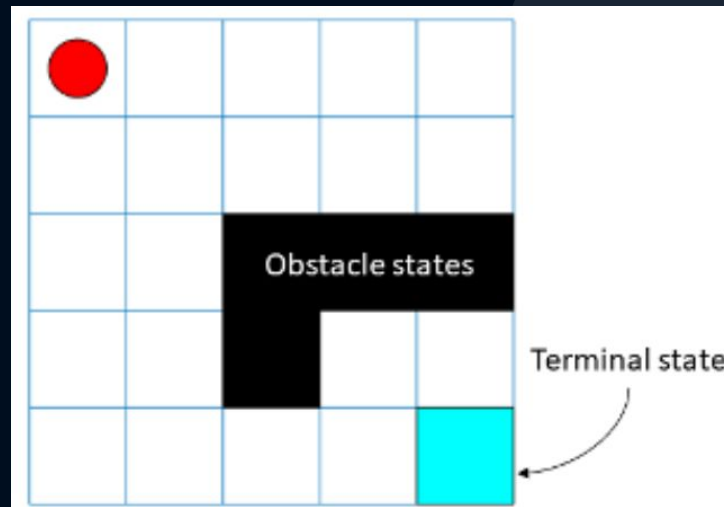
- (initial) state
- Action (chosen by agent's policy)
- Reward (environment)
- Next state (environment)
- Action
- ...

Problem Setting

- State Space S
- Action Space A
- Reward Function $R(s,a)$
- Transition Function $P(s' | s,a)$
- Objective Expected sum of rewards

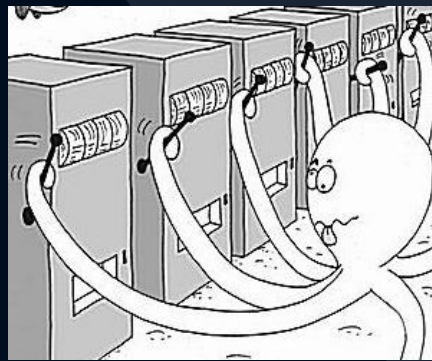
Example: Gridworld

- Example:
 - Environment: Gridworld.
 - State: Current grid position.
 - Actions: Up, Down, Left, Right. (unless obstacles)
 - Reward: +10 for reaching terminal state, -1 for each step.
- Design rewards for goal of reaching terminal state as quick as possible



Simpler Example: Multi-armed Bandit

- The 'game' only has one state, with multiple fixed actions
- Each action has its own reward, potentially drawn from some distribution
- We have to choose one action to pick that gives us the most (expected) reward, but we can play many times
- Examples:
 - Recommendation feeds (tiktok)
 - Choosing a restaurant to eat at



Approaching the Bandit Problem

- We would like to figure out which actions are the most fruitful (long term gain), while also using our turns to take actions that we already know are fruitful (short term gain)
- This tradeoff between *exploration* and *exploitation* is a general tradeoff that appears all throughout RL
- Biphasic Approach
 - randomly choose for some predetermined amount of steps
 - afterwards only pick the empirical best
- Epsilon-greedy
 - With some small probability, take a random action
 - Otherwise pick the action with the best average history of rewards

Bandits in Pokerbots: Learning across hands

- We have a collection of bots with different behaviors
- At each hand (or run of 25 hands), we pick a bot to use to decide the action
- Leads to some random reward (chip delta)
- Finding the bot that does well against our particular opponent across hands can help us counter them with an exploitative strategy
- This can be good for building on existing bots that don't have an obvious one that's better, and don't carry state variables across rounds/runs



RL Paradigms

Some General Approaches

We want to define our agent behavior using an evaluating function, but what should this function input?

- Q-learning: actions
- Value Iteration/Approximation: states
- Policy Methods: agent's policy itself

Q-learning

- Let's return to thinking of poker as a multi-step process
 - Extensive form instead of matrix form in the game theory lecture
- Want to find a value function $Q(s, a)$ for each state-action pair representing how good it is to be in that state taking that action
- If it's accurate, then we simply choose the action that has the highest stored value!
- Q-learning is a sample-based, one-agent way to tabulate *the game states* of this process and *the quality of each action* we could take in any given game state

Q-table

Initialized

Q-Table		Actions					
		South (0)	North (1)	East (2)	West (3)	Pickup (4)	Dropoff (5)
States	0	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	327	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	499	0	0	0	0	0	0

Training

Q-Table		Actions					
		South (0)	North (1)	East (2)	West (3)	Pickup (4)	Dropoff (5)
States	0	0	0	0	0	0	0
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	328	-2.30108105	-1.97092096	-2.30357004	-2.20591839	-10.3607344	-8.5583017
	.	.	.	.	.	.	.
	.	.	.	.	.	.	.
	499	9.96984239	4.02706992	12.96022777	29	3.32877873	3.38230603

Update rule

$$Q^{new}(s_t, a_t) \leftarrow (1 - \alpha) \cdot \underbrace{Q(s_t, a_t)}_{\text{old value}} + \underbrace{\alpha}_{\text{learning rate}} \cdot \overbrace{\left(\underbrace{r_t}_{\text{reward}} + \underbrace{\gamma}_{\text{discount factor}} \cdot \underbrace{\max_a Q(s_{t+1}, a)}_{\text{estimate of optimal future value}} \right)}^{\text{learned value}}$$

The upsides of Q-learning

- *General*: Q-learning can learn a policy to maximize final reward even if rewards happen incrementally
- *Simple and intuitive*: we repeatedly play games, and we take the deterministic action with the highest quality (do what worked well in the past)
- *Theoretically sound*: for any finite Markov decision process, Q-learning finds a policy that maximizes expected reward
- Model free!

Crucial downsides

- Slow to converge
- Prone to getting stuck in local optima
- Intractable if the state-action space is too large
- What do we do?
 - Use neural networks as an approximation (DQN); Q-table $\rightarrow$ Q-NN
 - Use a specialized algorithm for large, imperfect information games

Value Iteration

- Instead of a function for every state-action pair, seek a function $V(s)$ that reports the value of being in a given state
- To choose actions, we try to pick one that leads to the highest next state (or average if nondeterministic transitions)
- This can save compared to Q-learning if action space is large
- However it still faces many of the other downsides in Q-learning

Policy Methods

Direct Policy Optimization:

- Our behavior is directly governed by a *policy* $\pi(a|s)$ that tells us how to (possibly randomly) choose an action from a given state.
 - `Player.get_action`
- If we believe there's some idea of correct actions, then we can apply supervised approaches

Actor-Critic Methods:

- Actor: Updates policy.
- Critic: Evaluates policy using value function.

Lunch 🤪

Leave any type of feedback at pkr.bot/feedback!



INTERACTIVE CODING:

pkr.bot/q

SOLUTIONS:

pkr.bot/qs



FREE TO ENTER

SoCal College Poker Tournament

Hosted by Harvey Mudd College Poker

APPLICATIONS CLOSE
February 8, 11:59 PM PST



PRIZE POOL
\$10K+
IN CASH PRIZES



EXCLUSIVE
NETWORKING
WITH SPONSORS



2-DAY
MTT
NO-LIMIT HOLD'EM

Tournament Details

When
March 7-8 2026

Where
Los Angeles

Entry Fee
100% FREE

Eligibility
US College Students

PROUD SPONSORS

GOLD SPONSORS

Jump
TRADING

HRT

CITADEL

SILVER SPONSOR

Jane Street

BRONZE SPONSORS

GTS

SIG
SUGARMAN

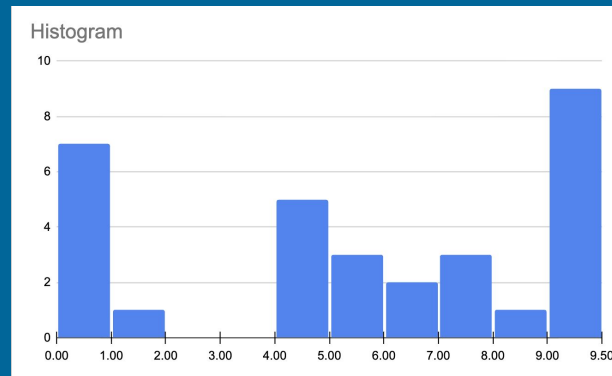
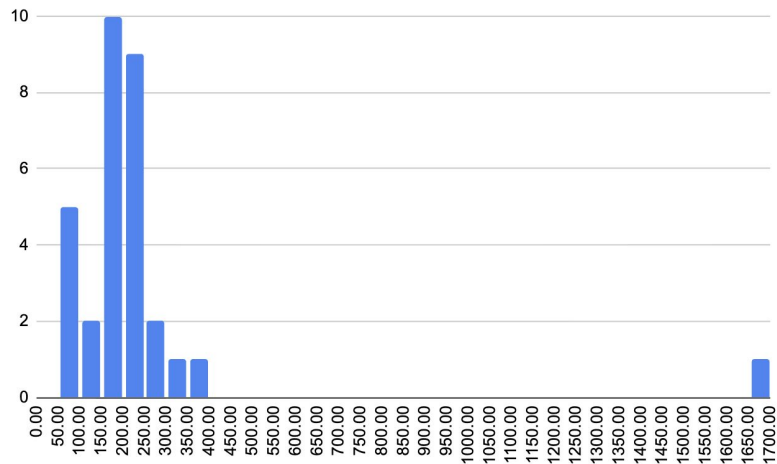
Apply @ socalcollegepoker.com

Harvey Mudd College Poker • Southern California

socalcollegepoker.com
March 7-8

Apply by February 8th

Sum Game: kerb *nbhakar*



Answer: 158

Winning guess: 162

Thanks for watching!

Information hub at pkr.bot/hub

Slides/notes will be posted on pkr.bot/resources

Make sure to check pkr.bot/piazza for updates

Lecture recordings at pkr.bot/panopto

Leave feedback at pkr.bot/feedback!